STUDENT PROJECT WORKBOOK

Modular Dataset Profiler & Cleaning Pipeline

PYTHON SOFTWARE DEVELOPMENT | MODULES, PACKAGES & FILE IMPORTS

> YOUR MAIN MISSION: As a team, build ONE working Python project that reads a CSV dataset, profiles its missing values, fills those missing values using simple rules, and exports a cleaned CSV. The program must be split into properly named Python modules inside a package. The key challenge is making the files work together—not building advanced data science algorithms.

Group name: _____ Members: _____

Sprint date: _____ Submission date: _____ Total: 20 marks

1. Project overview & real-world relevance

A data engineer often receives incomplete data that must be checked and prepared before it can be used. In real projects, one enormous script is hard to test, reuse and fix. Teams instead separate tasks: one file loads data, one examines it, one cleans it, and one saves it. This is called modular design.

In this project you will use a small customer-orders CSV. Pandas will do only straightforward tasks. Your biggest achievement is passing the same DataFrame through a clear sequence of files.

What success looks like

- Run the project from main.py in the project root.
- Read the source CSV into a Pandas DataFrame; show a preview and the missing-value count.
- Replace missing values according to the stated rules, then save cleaned_orders.csv.
- Keep loading, profiling, cleaning and exporting in four separate modules.

2. Prerequisite knowledge

You should already know these basics. A complete beginner may pair with a teammate for the first stage.

- Create folders and .py files in VS Code or another Python editor.
- Create and call basic Python functions, pass arguments and return values.
- Understand variables, strings, lists and basic dictionaries.
- Understand what a module is, and use import or from ... import.
- Use simple Pandas operations on a DataFrame: read_csv(), head(), isnull().sum(), fillna(), to_csv().

Beginner support: A DataFrame is a table with rows and columns held in Python. A CSV is a plain-text table file. A module is one .py file; a package groups related modules. You do not need machine learning, complex statistics, classes or sophisticated Pandas transformations.

3. Core concepts you will practise

Concept	How you demonstrate it
Packages and __init__.py	Create a pipeline folder containing __init__.py and four modules.
Importing local modules	Import each needed pipeline function into main.py.
Function inputs and outputs	Pass a DataFrame into functions and return it when the next stage needs it.
Separation of concerns	Keep every module responsible for exactly one stage.
Simple data profiling	Display a preview, dataset dimensions and missing-value counts.
Simple data cleaning	Fill missing values using the specified fixed rules.
File output	Save a CSV that can be opened and checked.

4. THE MAIN PROJECT QUESTION

Design, build and demonstrate a Python application named “Modular Dataset Profiler & Cleaning Pipeline”. The application must load the provided customer-orders dataset, inspect its first five rows and missing values, replace missing values by column, and export a cleaned dataset. Organize the application into the exact package-and-module structure shown below. Use main.py to import and coordinate the four stages so that a Pandas DataFrame moves from one stage to the next. Submit the working project and its final CSV output.

4.1 Required project architecture

```
PROJECT FOLDER BLUEPRINT • NOT PROGRAM SOURCE CODE
modular-dataset-pipeline/
main.py
requirements.txt
README.md
data/
orders_raw.csv
output/
cleaned_orders.csv (created when you run your app)
pipeline/
__init__.py
ingestion.py
profiler.py
janitor.py
exporter.py
```

The __init__.py file may be empty. Its purpose in this assignment is to make pipeline an explicit regular Python package. The output directory may be empty until you run the project.

4.2 One DataFrame travelling through the project

```
DATA FLOW • CONCEPTUAL DIAGRAM, NOT PYTHON CODE
Raw CSV
↓
ingestion.py → DataFrame
↓
profiler.py → Report preview & missing values
↓
janitor.py → Cleaned DataFrame
↓
exporter.py → cleaned_orders.csv
```

The profiler should inspect the DataFrame without replacing it. The cleaning stage should return a DataFrame so that main.py can pass it into the exporter. Choose meaningful function names and keep them consistent across imports.

5. Specific responsibilities of every file

File	Required responsibility	Success check
pipeline/ingestion.py	Define a function that loads data/orders_raw.csv using Pandas and returns a DataFrame.	main.py can receive a DataFrame.
pipeline/profiler.py	Define a function that receives a DataFrame, displays the first 5 rows, reports row/column count and missing-value totals per column.	The terminal shows a useful report.
pipeline/janitor.py	Define a function that receives a DataFrame, fills missing customer/city values with “Unknown”, fills missing orders/amount values with 0, and returns the cleaned DataFrame.	No missing values remain in these columns.
pipeline/exporter.py	Define a function that receives the cleaned DataFrame and saves it as output/cleaned_orders.csv without an extra index column.	A usable CSV appears in output/.
main.py	Import and call the functions in the required order; briefly label each stage when it runs.	One run starts and finishes the pipeline.
pipeline/__init__.py	Keep this package marker present; no special logic is required.	Package structure is correct.

5.1 Rules for the source dataset

Use the accompanying orders_raw.csv. It contains five columns: record_id, customer, city, orders and amount. Some cells are intentionally blank. Treat record_id as an identifier and do not overwrite it. There are 10 data rows in the provided file.

Column	Data type / meaning	Missing-value rule
record_id	Unique record number	Leave unchanged
customer	Customer name (text)	Replace missing with Unknown
city	City (text)	Replace missing with Unknown
orders	Number of orders (number)	Replace missing with 0
amount	Total amount (number)	Replace missing with 0

Do not delete rows or build complex cleaning algorithms. Use basic Pandas operations only. If your team creates a different dataset, it must keep these columns, and your rules must still work.

6. Hands-on sprint instructions

Stage A — Set up the workspace

- Create the root folder, package folder, data folder and output folder using the exact names shown.
- Create all five Python files and __init__.py. Install Pandas in your environment and record it in requirements.txt.
- Place orders_raw.csv inside data/. Open it and confirm the five column headings.

Stage B — Build one module at a time

- Start with ingestion.py. Make it load the CSV and return the resulting DataFrame.
- Create profiler.py to display head(), the number of rows/columns, and isnull().sum().
- Create janitor.py to apply only the four simple missing-value rules.
- Create exporter.py to save the cleaned table in output/.

Stage C — Connect the modules

- In main.py, import the functions from the pipeline package and call them in order.
- Pass the loaded DataFrame into profiling and cleaning. Pass the cleaned result into exporting.
- Run main.py from the project root and fix any import or file-path errors.

Stage D — Verify and demonstrate

- Show the original missing-value counts, then the cleaned dataset’s missing-value counts.
- Open the exported CSV to verify it has the same 10 records and expected five columns.
- Have each member explain the responsibility of one file and where the DataFrame travels.

7. Acceptance tests: how to know it works

Test	Expected result
Run main.py from the root folder	The project completes without crashing.
Inspect the pre-cleaning profile	Shows 10 rows, 5 columns and some missing values.
Inspect the cleaned output	All missing values in customer, city, orders and amount are filled.
Count output records	Still 10 data rows; none deleted.
Check exported CSV columns	record_id, customer, city, orders, amount; no extra unnamed index.
Check modularity	Removing an import should break a needed stage; no stage logic should be duplicated in main.py.

8. Team roles and collaboration

Suggested roles for groups of four: Pipeline Architect (folder structure and imports), Ingestion & Profiling Developer, Cleaning & Export Developer, and Integration & Test Lead. For smaller groups, combine roles. Every member should still be able to explain the full data flow.

> Team checkpoint: Before submitting, each group member should answer: “Which file do I change if CSV loading fails?”, “Which function receives the DataFrame next?”, and “Why is our project easier to maintain than a single long script?”

9. Expected deliverables — submit ALL of these

• A GitHub repository link containing the required files and folders.

• The source CSV at data/orders_raw.csv.

• The generated cleaned CSV at output/cleaned_orders.csv.

• A README.md containing the project goal, team members, installation/run steps, and a short description of each module.

• A requirements.txt listing the dependency used for the project.

• A terminal screenshot or short demonstration showing the profiler and a successful run.

• A brief 3–5 sentence group reflection describing one import/module problem you solved.

Submission format: one repository link per team. Make sure your instructor can run the project from the root directory following your README.

10. Scoring rubric — 20 marks

Area assessed	Marks	What earns full marks
Required folders and files	4	Exact structure, including package marker and clearly named modules.
Correct custom imports	4	main.py imports and calls all four stages without import errors.
DataFrame passed between modules	4	Function arguments/returns and stage order are correct.
Basic profiling and cleaning	3	Preview, null counts, simple rules; no unnecessary complexity.
Correct CSV output	2	Cleaned file saved to output/ with correct records and columns.
README, setup and reproducibility	2	Another learner can install and run the project.
Team demonstration/reflection	1	All members can explain their contributions and the data flow.
TOTAL	20	Architecture, imports and DataFrame flow account for 12 marks.

11. Troubleshooting hints (without solutions)

If you see...	Check this first
ModuleNotFoundError	Is pipeline/ spelled correctly? Does it contain __init__.py? Are you running from the project root?
FileNotFoundError	Is data/orders_raw.csv in the expected folder? Are you running main.py from the root?
AttributeError or NoneType errors	Did a function return the DataFrame when the next function needs it?
Wrong or missing cleaned output	Did you pass the cleaned DataFrame, not the original one, into the exporter?
Unexpected extra column	Check the CSV export setting for including the DataFrame index.

12. Final submission checklist

☐ Our repository has the exact required package structure.

☐ main.py runs the whole pipeline in the right order.

☐ Our modules do not duplicate one another’s responsibilities.

☐ The original file is preserved and the cleaned CSV exists.

☐ All required missing values are handled.

☐ Our README explains how to run the project.

☐ Every group member can explain at least one module and the DataFrame flow.

Official reference material: Python modules/packages (https://docs.python.org/3/tutorial/modules.html) and Pandas CSV/DataFrame operations (https://pandas.pydata.org/docs/).

What are Python libraries, modules & packages?

Before you start your project, understand the tools you are going to use. A library is reusable code that helps programmers complete familiar tasks. You often use a library by importing a module, then calling one of its functions. You do not need to memorize every function: you need to know how to read the documentation and find the functions your project requires.

Term	Meaning	Example that applies to this workbook
Standard library	Modules distributed with Python. Normally no additional pip installation.	Python pathlib or json (depending on your task)
Third-party package	Software installed separately, usually with pip.	A separately installed package when a task calls for it
Custom module	A .py file your own team creates.	main.py and another .py file created by your team
Custom package	Your folder of related Python modules.	your project package folder containing __init__.py

Start with these general resources - required for both departments

Python Tutorial - Modules & Packages — Learn how modules work, how files import one another and why __init__.py is used.
https://docs.python.org/3/tutorial/modules.html

Python Standard Library Overview — Learn how to recognize modules that already come with Python.
https://docs.python.org/3/library/index.html

Python Packaging Guide - Installing Packages — Learn how pip and virtual environments work when a third-party package is actually required.
https://packaging.python.org/en/latest/tutorials/installing-packages/

How to read documentation without getting lost

• Step 1: First read the two-sentence explanation of the library. Ask, “What problem does it solve?”

• Step 2: Find the one or two functions your own project needs. Read the input parameters, return value and example.

• Step 3: Test a tiny example separately before placing the function in your own module.

• Step 4: Explain to a teammate what you passed into the function and what came back.

• Step 5: Add the library name and installation method (if any) to your README.

> READING RULE: Complete Part A first. After that, read only the section for your own department - and for Cybersecurity, only your selected Option A, B or C. The resource section teaches tools; the assignment section tells you what to build.

PART B / DATA SCIENCE & DATA ENGINEERING ONLY

Learn Pandas before building the pipeline

YOUR PROJECT: Modular Dataset Profiler & Cleaning Pipeline. There is ONE required third-party library: pandas. You will also use your own modules in pipeline/. No cybersecurity libraries or hashing tools are required for this assignment.

1. What is pandas, and why are you using it?

pandas is a Python library for working with structured data such as tables in CSV files. A DataFrame is an in-memory table: rows represent records, and columns represent fields. Your project uses pandas to load a table, inspect missing cells, replace them and save a clean table. You are NOT expected to do machine learning, charts, forecasting or complicated statistics.

First read: Pandas - 10 minutes to pandas — Introduces DataFrames and the simple table operations you will need.
https://pandas.pydata.org/docs/user_guide/10min.html

Pandas - Getting Started — A beginner guide to what pandas is and what DataFrames can do.
https://pandas.pydata.org/docs/getting_started/index.html

2. Installation and the first tiny example

pandas does not come with standard Python installations. Install it in your project environment using pip. Record the dependency in requirements.txt and explain installation in README.md.

INSTALLATION COMMAND (RUN IN TERMINAL)

```
python -m pip install pandas
```

SMALL LEARNING EXAMPLE (NOT A PROJECT SOLUTION)

```
import pandas as pd
small_table = pd.DataFrame({"item": ["Pen", "Book"], "qty": [2, 3]})
print(small_table.head())
```

Explanation: import makes pandas available under the short name pd; DataFrame creates a table; head() displays the beginning of that table. This is a learning example, not your full pipeline.

3. Understand the exact Pandas functions needed

Project stage	Function / attribute	Meaning in plain language
Ingestion	pd.read_csv(path)	Read a CSV file into a DataFrame. The result can be passed to another function.
Profiling	df.head()	Display the first five rows.
Profiling	df.shape	A pair: number of rows and number of columns.
Profiling	df.isnull().sum()	Count the missing values in each column.
Cleaning	df.fillna(value)	Replace missing cells with a chosen value; values can differ by column.
Export	df.to_csv(path, index=False)	Write a CSV file without adding the DataFrame index as a new column.

4. Simple examples to understand BEFORE writing your pipeline

SMALL LEARNING EXAMPLE (NOT A PROJECT SOLUTION)

```
import pandas as pd
practice = pd.DataFrame({"city": ["Lagos", None, "Kano"]})
print(practice.isnull().sum())
print(practice.fillna("Unknown"))
```

Explanation: None represents a missing value here. isnull().sum() reports how many entries are missing. fillna("Unknown") returns a table where the blank city is replaced. In your project, text and numeric columns have different replacement rules.

5. Official documentation - directly tied to your modules

read_csv() - ingestion.py — Arguments, CSV path and resulting DataFrame.
https://pandas.pydata.org/docs/reference/api/pandas.read_csv.html

DataFrame.head() - profiler.py — Previewing records.
https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.head.html

DataFrame.shape - profiler.py — Counting rows and columns.
https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.shape.html

DataFrame.isnull() - profiler.py — Identifying missing values.
https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.isnull.html

DataFrame.fillna() - janitor.py — Choosing replacements for missing values.
https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.fillna.html

DataFrame.to_csv() - exporter.py — Writing the cleaned CSV.
https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.to_csv.html

6. How your team should study

☐ Every learner should explain what a DataFrame is.

☐ Before building ingestion.py, practise read_csv() on the provided orders_raw.csv.

☐ Before building profiler.py, practise head(), shape and isnull().sum().

☐ Before building janitor.py, practise fillna() on a tiny practice table.

☐ Before building exporter.py, learn why index=False matters.

☐ Finally use your custom imports to pass the DataFrame through the four files.

> You do not need NumPy as a separately imported tool, visualization libraries, database drivers, security packages or hashing libraries for the scope of this task. pandas itself installs its dependencies as needed.

PART C / REQUIRED FOR EVERY TEAM

GitHub submission method - REQUIRED

> SUBMIT ONLY ONE GITHUB REPOSITORY LINK PER TEAM. The repository must contain the full working project, not merely screenshots, a PDF, a ZIP attachment or a link to an individual .py file. Use the instructor-provided submission channel and deadline.

Step-by-step: from your computer to GitHub

• 1. Finish your project locally. Run main.py from the root folder and check all required outputs.

• 2. Create a GitHub repository for your team. Give it a meaningful name; choose public visibility unless the instructor requests private.

• 3. Upload/PUSH the complete project folder to the repository using Git commands or GitHub Desktop. Include the required package files, sample data and README.

• 4. Open the repository page in a browser. Confirm all folders and files are visible online and that the latest updates were pushed.

• 5. Copy the repository homepage URL (format: https://github.com/username/repository-name). This - not a file link - is the submission link.

• 6. One representative submits the URL with team name, department/track, member names and chosen project where relevant.

• 7. Check link permissions. If the repository is private, give the instructor access before submission; a link alone will not grant access.

Official GitHub how-to resources

GitHub Docs: Add locally hosted code to GitHub — creating the repository and pushing local changes
https://docs.github.com/en/migrations/importing-source-code/using-the-command-line-to-import-source-code/adding-locally-hosted-code-to-github

GitHub Desktop: Add an existing project — a visual alternative if the Git command line is unfamiliar
https://docs.github.com/en/desktop/adding-and-cloning-repositories/adding-an-existing-project-to-github-using-github-desktop

Paste this information into the submission message

Field	Required value
Department	Data Science & Data Engineering
Group name	Your team name
Team members	Names of all contributors
Project selected	Data pipeline
GitHub repository URL	https://github.com/username/repository-name
Confirmation	We pushed the complete project and confirmed the link opens.

Before clicking submit

☐ The link opens the REPOSITORY homepage and all files are visible.

☐ README.md has the goal, team members, setup/run instructions, and module responsibilities.

☐ No passwords, secrets, private keys, actual keystrokes, personal data or unsafe files were committed.

☐ Required output/sample evidence is present; someone else can follow README to run the app.

The original grading rubric remains out of 20 marks. GitHub submission and documentation are part of the existing deliverables and README/reproducibility requirements; they do not add extra marks.